Rock, Paper, Scissors, Scalpers

CSE 6730: Modeling & Simulation — Spring 2026

Group 10

Denys Chernenko
Smaran Iyer
Adrian Lozada

https://github.gatech.edu/alozada7/cse6730-project

Final Report — May 1, 2026

Final: Rock, Paper, Scissors, Scalpers

github.gatech.edu/alozada7/cse6730-project.git

Group 10: Denys Chernenko, Smaran Iyer, Adrian Lozada

1 Abstract

We present an agent-based simulation of a collectible card game economy in which players compete using Rock-Paper-Scissors mechanics and scalpers trade cards for profit. Two interacting agent populations are modeled: players who build decks to maximize win-rate given budgets and card availability, and scalpers who exploit price movements driven by shifting player demand. Card values evolve through a shared order-book market, where prices respond to net buy and sell pressure each tick. To ground the agent-based dynamics in theory, we derive a mean-field ordinary differential equation baseline using replicator dynamics, which predicts the long-run equilibrium distribution of card strategies under idealized conditions. We compare this analytical baseline against the full agent-based model to measure how scalper activity distorts strategy evolution and market efficiency. Experiments vary scalper population size, strategy type, and market parameters to investigate how non-player participants affect price volatility, deck inequality, meta stability, and overall market welfare.

Key terms used in this paper:

Table 1: Key terms used in this paper.

Term	Meaning
ACE	Agent-based Computational Economics
RPS/Card	ROCK (R), PAPER (P), or SCISSOR (S)
Deck	Player or scalper inventory
Market price	Cost to buy some card
Buy order	Request to buy some card (can be above/below market price)
Sell order	Request to sell some card (can be above/below market price)
Limit price	Max buy / min sell price
Tick/Step	One round of play + market activity
Scalper	Profit-seeking trader; does not play

2 Project Description

Rock, Paper, Scissors, Scalpers is rooted in Agent-Based Computational Economics (ACE)[9]. ACE methods study the behavior of large, complex economies that arise from the interactions of many independent decision making agents acting autonomously. Tesfatsion notes that economists have traditionally tried to explain global or macroeconomic trends via common knowledge assumptions, fixed rules, or imposed market constraints, but these types of "top-down" models do not adequately represent real life as real life does not have a global rule-making force. Instead, ACE methodology takes a different approach: ACE models consist of individual participant (agent) populations that have similar initial states but eventually evolve to unique end states due to interactions with other agents and changes in the economic system they participate in.

This project models a card trading market built on top of a competitive Rock-Paper-Scissors (RPS) game. There are two kinds of participants: players who compete in RPS matches against each other, and scalpers (automated trading bots) who buy and sell cards to profit from price swings caused by those matches. The goal is to study how different trading strategies perform against each other in a small, controlled market where prices are driven by real game activity, not random noise.

The simulation follows the ACE framework. Each participant is an independent agent that follows its own rules, and the interesting behavior emerges from all of them interacting. There is no single controller; the market price and card distribution are outputs, not inputs.

The scalper portion of the simulation is inspired by financial markets — specifically how short-term traders (scalpers) behave in a liquid market where prices fluctuate based on supply and demand. Financial markets are driven by:

(1) **Supply and demand** — prices rise when more people want to buy than sell, and fall when more

want to sell.

(2) **Trading strategies** — some traders follow trends, some try to detect when a price has fallen too far and will bounce back, some try to exploit tiny price gaps.

(3) **Market microstructure** — the rules of how orders are submitted, matched, and settled affect outcomes in non-obvious ways. In a real stock market, these forces produce complex, emergent price patterns that are hard to predict. This simulation captures the same mechanics in a simplified, transparent setting so the effects of different strategies can be studied cleanly.

Why RPS cards? Each of the three card types (ROCK, PAPER, SCISSOR) has its own price and supply. Because one card beats another in the game, players have a reason to prefer certain cards. This creates demand that scalpers can try to detect and profit from.

2.1 Novelty of Our Approach

Based on our findings, existing research focuses on only one side of this problem at a time. On the market side, models like [4] and [1] study how prices form and how agents trade, but their agents have no underlying game motivating their demand valuations are fixed or exogenous, not driven by a shifting competitive meta. On the game dynamics side, [8] models how cyclic dominance shapes strategy evolution through replicator dynamics, but there is no market layer: strategies spread through imitation and selection, not through buying and selling under budget constraints. The ACE framework [9] gives us the bottom-up methodology we build on, but to our knowledge, no ACE application has coupled a cycling game meta directly to a price-forming market. Scalping has been studied in ticket sales [2] and futures markets [7], where resellers exploit timing gaps between supply and demand, but it remains an underexplored topic in markets where demand is player-driven and shifts based on game outcomes. Our simulation sits at the intersection of all of these: scalper agents operate in the price layer while distorting game outcomes through card availability, creating a feedback loop where meta shifts drive scalper behavior, which in turn reshapes the meta.

3 Literature Review

3.1 Foundation

Tesfatsion notes that economists have traditionally tried to explain global or macroeconomic trends via common knowledge assumptions, fixed rules, or imposed market constraints, but these types of "top-down" models do not adequately represent real life as real life does not have a global rule-making force. Instead, ACE methodology takes a different approach: ACE models consist of individual participant (agent) populations that have similar initial states but eventually evolve to unique end states due to interactions with other agents and changes in the economic system they participate in. This is what Tesfatsion refers to as a "bottom-up" method of economic modeling. In this approach, the model has the following components:

(1) An initial population of agents and resources.
(2) Each agent has its own internal state: inventory, budget, and knowledge of the system.
(3) Decision making rules: How an agent will act given their state and observations.
(4) Interaction rules: Who trades with whom, how do prices form, how information spreads.

Using these components, the model is executed as a sequential time-based simulation, and the resulting behavior is analyzed as an outcome of the rules and interactions between the agents. In our project, we feature two independently-motivated agent populations:

(1) **Players**: Game motivated agents who buy cards and build decks with the intent to maximize winrate given budgets and card availability.
(2) **Scalpers**: Profit motivated agents who trade to maximize investment returns given price changes and anticipated shifts in player demand.

3.2 Efficiency of Agent-Market Simulation

Gode and Sunder tested the efficiency of agent-based markets directly against human markets by running both groups on identical market configurations [4]. Efficiency here refers to allocative efficiency, or the ratio of realized total surplus to the theoretical maximum, where high efficiency means goods are being directed to their highest-value use [4]. The agents were "Zero-Intelligence" (ZI), storing no information about market state or trade history and instead submitting random bids and asks. In the first experiment, ZI-Unconstrained (ZI-U) agents could execute any trade they had resources

for, including unprofitable ones, and this market showed poor allocative efficiency that did not reflect human behavior. In a second experiment, ZI-Constrained (ZI-C) agents were introduced with a single budget constraint: they could not sell below cost or buy above value. This one constraint was sufficient for the simulation to reach efficiency levels comparable to, and in some cases matching, the human market. The result challenges the traditional view that market participants must be rational or informed, showing that even minimal rules in a simulated agent market can reproduce aggregate outcomes observed with humans.

3.3 Simulation

Brock and Hommes investigate what happens when traders in a financial market have different beliefs about where prices are going. Some traders are *fundamentalists* who think prices should reflect its "true value", and others are *chartists* or "noise traders" who chase trends [1]. The "true value" is calculated by summing all future payments (dividends) it will generate, discounted back to today or its present discounted value (PDV). The agents switch between strategies based on recent perfomance. The key takeaway is that markets don't need external shocks to get extreme price swings. The interaction between different belief types is enough to generate overvalued assets (bubbles), crashes, and unpredictable dynamics in the market.

Our players (agents) will have different strategies (trend-followers, random buyers) and the scalpers will have their own profit-seeking logic. The way in which our players could choose strategies is by first updating their price of the cards using the cards' Present Discounted Value. For instance, a cards that wins 50% of games is worth more than one winning 20%.

3.4 Cyclic Dominance in Evolutionary Games

In "Cyclic dominance in evolutionary games: A review" by Szolnoki et. al, we see a backbone of what our project is trying to do: understand how closed loops of dominance generate dynamic, shifting equilibria [8]. Mean-field models predict cycles, spatial models produce spirals, and finite populations can lead to "survival of the weakest". Our deck-building meta is a system of cyclic dominance system embedded inside a market. The paper also makes a distinction between well-mixed and structured populations, which maps to centralized versus fragmented marketplaces in our project. We can use replicator dynamics (strategy frequency chances based on payoff) to model how card popularity shifts, and then use finite-population stochasticity to capture sudden crashes when a strategy gets overbought.

In this paper, biodiversity is preserved or destroyed depending on mobility, mutation, and interaction structure. Scalpers are like a "new species" that do not compete in matches but alter resources availability. In evolutionary terms, they change the payoff landscape and effective reproduction rates of strategies. We can model scalpers as agents who remove cards from circulation, altering supply and therefore the cost function, which in turn feeds back into strategy viability. That introduces a coevolutionary cycles, where meta shifts -> scalpers respond -> supply changes -> win rates shift again. The paper discusses phase transitions and oscillatory regimes, which suggests that we may see tipping points in our project. For example, a certain player-to-scalper ratio might push the system from stable cycling to dominance collapse, where one strategy breaks the loop.

3.5 Economics of ticket resale

Resale markets naturally appear when goods are limited and people value them differently over time. In Pascal Courty's "Some Economics of Ticket Resale", event organizers sell tickets early, but some buyers later realize the value attending more than others [2]. This timing difference creates an opportunity for brokers to buy early and resell later at higher prices. That logic maps to our project. In our game, cards (rock, paper, scissors) are limited in supply, and their value changes as the meta evolves. Players who anticipate future demand can buy strong cards early, while others may only realize a card's value once it becomes dominant. Scalpers step into that gap, buying scarce "meta" cards and reselling them when their price rises. Courty's model helps explain why scalpers might persist even if the game designers try to discourage them: as long as resale is possible and demand changes over time, profit opportunities will exist.

We can use Courty's framework to guide both our simulations and analysis. His model distinguishes between resellers who simply extract profit and those who potentially add value by improving market efficiency. In our project, we can test both possibilities. Do scalpers stabilize the market by signaling which cards are strong and moving cards toward players who value them the

most? Or do they destabilize the meta by hoarding dominant cards and creating artificial scarcity? By varying the number of players, the number of scalpers, and the speed at which card prices adjust, we can observe how the meta shifts and whether scalpers increase or decrease overall welfare. Courty's work gives us the following baseline: if timing differences and resale are enough to sustain brokers in ticket markets, we should expect similar dynamics in our card trading game, and we can measure how those dynamics reshape strategy and competition.

4 Conceptual Model

The simulation operates as a discrete-time system in which time advances in fixed steps called *ticks*. Each tick, players compete in RPS matches and all agents interact with a shared card market. The model has three core components: a *tick loop* that sequences events within each step, a *market* that mediates card exchange and price formation, and two *agent populations* (players and scalpers) that follow distinct decision rules. A mean-field ODE serves as the baseline for the agent-based model by predicting the long-run equilibrium card distribution under idealized conditions. Using this baseline, we can evaluate the effects of the market and the inclusion of scalpers on player behavior and global card populations.

4.1 Ticks

Each tick proceeds in six stages:

(1) **Match Players:** Shuffle the order of all players, split players into two halves, shuffle one half, and pair them 1-to-1.
(2) **Build Games:** Create a Rock–Paper–Scissors match for each pair.
(3) **Play Games:** Each player draws a card from their deck and plays it. Outcomes (win/loss/tie) are determined and recorded.
(4) **Agents Decide:** All agents observe prices and may submit buy/sell orders.
(5) **Market Clears:** Orders are processed: buy orders execute if price $\geq$ market price; sell orders always execute. Prices update by $\pm 5\%$ based on net demand, and holdings are updated.
(6) **Record State:** Prices, holdings, and results are saved.

4.2 9-Card RPS

Implementing a real-world trading card game such as "Magic: The Gathering", "Pokémon", or "Yu-Gi-Oh!" is prohibitively complex. Instead, we designed a simplified card game that captures the essence of playing games and building decks against other players. The game serves as a model for real market behavior. Agents hold cards (*assets*), the cards have prices driven by supply and demand, and profit-seeking intermediaries (scalpers) exploit card price movements.

We implement a two-player trading card game based on rock-paper-scissors. The game is played with no hand management. The rules are as follows:

- Each game is played between exactly two players. Multiple games run simultaneously per simulation tick.
- Each player holds a deck of exactly 9 cards, composed of some mixture of Rock, Paper, and Scissors cards. Decks are shuffled at the start of each game.
- Each turn, both players simultaneously reveal the top card of their deck. Resolution follows standard RPS dominance: Rock $\succ$ Scissors $\succ$ Paper $\succ$ Rock.
- The winner of each turn scores 1 point; both cards are sent to discard.
- On a tie, both cards are returned to their owner's deck and the decks are reshuffled.
- The first player to reach 3 turn wins takes the game and receives 1 unit of money.
- If both decks are exhausted before either player reaches 3 wins, the game is declared a draw and no money is exchanged.

This design was chosen deliberately for several reasons. Since players flip cards with no in-game choices, the game outcome is purely a function of *deck composition*. Expected win rates can therefore be derived semi-analytically from the deck composition vector and the RPS payoff matrix, connecting directly to the replicator dynamics framework in [8]. The absence of in-game agency also makes games fast to resolve at scale. Finally, the tie-reshuffle rule prevents ties from depleting card resources, and combined with the first-to-3 structure, ensures that most games produce a decisive winner.

4.3 Market

The market maintains a price and an inventory for each of the three card types, all initialized at $1.00 with 1,000 units.

Price update rule: after each tick, if more units of a card are bought than sold, its price increases by 5%; if more are sold than bought, it decreases by 5%. Equal volume leaves the price unchanged.

Example (ROCK demand spike): Buyers submit 3 buy orders for ROCK, while 1 sell order is submitted. The net buy pressure is +2, so the ROCK price updates from $1.00 to $1.05 (+5%), while PAPER and SCISSOR remain at $1.00.

This mechanism creates a feedback loop: popular cards become more expensive, discouraging further buying and encouraging selling, which eventually pushes prices back down. This oscillatory behavior is what scalpers attempt to exploit.

4.4 Players

Players are RPS competitors. At simulation startup, each player starts with a collection of randomly selected cards while maintaining a max deck size (in our case, $R+P+S = 9$). During each tick, one game is played among each pair of players. For each game, Winners and losers are decided based on the 9-card RPS format noted above. After each game, every agent evaluates the outcomes of their previous games, and updates their deck depending on their Player Type. Two player types are implemented, and both are based on [8]:

Mutator: ignores the outcomes of previous games, and instead, it randomly picks a card from its deck, then "mutates" it into one of the two possible cards that complement it. For example, R may mutate into P or S. This is similar to real world experimentation, and is crucial to avoiding population collapse in the simulation.

Copycat: Evaluates its own performance against that of its most recent opponent and probabilistically imitates the opponent's deck composition. After each game, the player compares its own fitness (its expected or empirical win rate) to an estimate of the opponent's fitness given the current population. If the opponent appears to be doing better, the player adopts a copy of the opponent's deck with probability proportional to the fitness gap; otherwise, it keeps its current deck. In this way, successful strategies spread through the population via social learning rather than random exploration.

4.5 Scalpers

Scalpers are trading bots that never play Rock–Paper–Scissors; their objective is purely to maximize profit. We consider four strategies:

NaiveGambler (baseline). Each tick, selects a random card and a random price in $[\$0.50, \$2.00]$. This serves as a benchmark.

Shadow (mean reversion with trend filter). Shadow assumes that prices reverting below their recent average may rebound, but only within an uptrend. It tracks an exponential moving average (EMA):

$$\mathrm{EMA}_t = 0.3 \cdot p_t + 0.7 \cdot \mathrm{EMA}_{t-1}.$$

A buy signal occurs when price $p_t < \mathrm{EMA}_t$ and the EMA has been increasing over the last three ticks. Shadow avoids downtrends. It exits when the EMA turns downward, when price reverts to the EMA, or after 12 ticks.

QuickGrabber (micro-momentum). This agent exploits short-term momentum. If price increases by more than 5% over the past two ticks, it buys:

$$\frac{p_t - p_{t-2}}{p_{t-2}} > 0.05.$$

It exits at +4% profit, −3% loss, or after 5 ticks.

MarketMaker (fixed spread). With no open position, the MarketMaker selects the card with the largest price drop in the previous tick and buys, anticipating a rebound. It exits at ±4% price movement or after 8 ticks.

Simulation Outputs. After running for N ticks, the simulation records:

- **Scalper final balances:** cash on hand plus the market value of any cards held.
- **Player outcomes:** total wins, losses, and ties across all matches.
- **Price trajectories:** time series of prices for each card type.
- **Market stock levels:** evolution of available inventory for each card.

4.6 Strategy Comparison.

Table 2 summarizes the entry, exit, and maximum hold parameters for each scalper strategy.

Simulation Outputs. After running for N ticks, the simulation records:

Table 2: Scalper strategy entry, exit, and hold parameters.

Agent	Entry	Exit	Max Hold
NaiveGambler	Random	Random	Any
Shadow	$p <$ EMA (uptrend)	EMA reversal / revert	12
QuickGrabber	Momentum $> 5\%$	$\pm 4\%$	5
MarketMaker	Largest drop	$\pm 4\%$	8

- **Scalper final balances:** cash on hand plus the market value of any cards held.
- **Player outcomes:** total wins, losses, and ties across all matches.
- **Price trajectories:** time series of prices for each card type.
- **Market stock levels:** evolution of available inventory for each card.

5 Simulation

5.1 Baseline Population Dynamics ODE

Prior to agent-based simulation, we model the evolution of card populations using an evolutionary RPS system based on Szolnoki, et al. [8]. Since players are game motivated, they derive their decision making from their experiences with other players. The player implementation is based on [8]'s four reaction schemes represented by eqns 2.1, 2.2, 2.3, 2.4, and our work takes these and labels them **Predator-eats-Prey**, **Copycat**, **Duplicator**, **Mutator**, respectively. To best mimic real-life deck-building behaviors, our implementation **only** uses the **Copycat** and **Mutator** as they mimic real player behaviors, such as copying the decks of winning players, and swapping (mutating) cards in their decks as a form of exploration. Therefore our system of player reaction schemes are as follows, where A, B, C represent three species in the system:

$$AB \xrightarrow{z} AA, \quad BC \xrightarrow{z} BB, \quad CA \xrightarrow{z} CC \tag{1}$$

$$A \xrightarrow{\mu} \begin{cases} B \\ C, \end{cases} \qquad B \xrightarrow{\mu} \begin{cases} A \\ C \end{cases} \qquad \text{and} \quad C \xrightarrow{\mu} \begin{cases} A \\ B. \end{cases} \tag{2}$$

This reduces the population density equations to:

$$\frac{da}{dt} = a\,[z(b-c)] + \mu(b + c - 2a) \tag{3}$$

$$\frac{db}{dt} = b\,[z(c-a)] + \mu(a + c - 2b) \tag{4}$$

$$\frac{dc}{dt} = c\,[z(a-b)] + \mu(a + b - 2c) \tag{5}$$

Where a, b, c are the population fractions of Rock, Paper, Scissors s.t. $a + b + c = 1$. Our implementation uses the same rate constant conventions (z, μ) as [8]. z is the copycat rate, and μ is the mutation rate. Outside of market and scalper influences, this system represents the baseline, and we will attribute any deviations from this baseline to the presence of the market and scalpers.

Using the `scipy.integrate` `solve_ivp` ODE solver (Dormand-Price RK45: Explicit Runge-Kutta of order 4 and 5) [3], the baseline simulation is implemented in `ode.py` using the modified Szolnoki Equations above. RK45 is the default solver provided by `integrate` and it is an appropriate explicit solver for this use case because:

(1) The modified Szolnoki ODEs are quadratic in a, b, c.
(2) The system is continuous and non-stiff, and the adaptive step size cleanly handles the tight oscillations of the Szolnoki ODEs.
(3) It is a standard solver for solving ODEs and population dynamics simulation. [5] [6]

Outside of market and scalper influences, this system represents the baseline, and we will attribute any deviations from this baseline to the presence of the market and scalpers.

5.2 Player Decision Making

Each `Player` agent keeps track of a deck $\mathbf{d} = (d_R, d_P, d_S)$ of card counts, a money balance, and a list of past game results. Every tick, the simulator calls `wake_up_agent` on the player, which then runs the player's `update_state` method. We implemented two player behaviors,

Mutator and **Copycat**, which are the agent-level versions of the reactions in Eqs. (2) and (1). Both behaviors are random: each tick they fire with a probability that depends on the ODE rates μ and z and the number of players N, so that the average rate of deck swaps in our simulation lines up with the rates used in the ODE.

Both behaviors share two helper computations. The first turns a deck into a frequency vector:

$$\mathbf{x}(\mathbf{d}) \;=\; \frac{\mathbf{d}}{\sum_i d_i} \tag{6}$$

defaulting to $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ if the deck is empty. The second estimates how "good" a deck is against the current population. Using the population card counts $\mathbf{c}$ and the RPS payoff matrix:

$$P \;=\; \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix} \tag{7}$$

the default fitness estimate (`FitnessModes.EXPECTED`) is

$$f(\mathbf{d}; \mathbf{c}) \;=\; \mathbf{x}(\mathbf{d})^\top P\, \mathbf{x}(\mathbf{c}) \tag{8}$$

which is just the expected payoff per game of a deck $\mathbf{d}$ played against a random card drawn from the population. We also have an `EMPIRICAL` mode that uses the player's win rate over the last $W = 50$ games (`FIT_WINDOW_SIZE`), but we use `EXPECTED` as the default because it matches the mean-field assumption of the ODE.

5.2.1 Mutator The Mutator is our agent-level version of the mutation reaction in Eq. (2), where a card in a player's deck gets randomly swapped for a different card. Each tick, the agent samples a value in $(0.0, 0.99)$ and acts with probability

$$p_{\text{mut}} \;=\; \frac{1}{\mu N}, \tag{9}$$

where N is the number of player agents (`self._sim_players`) and μ comes from `ODE_PARAMS`. With our settings $\mu = 0.01$ and $N = 500$, this gives $p_{\text{mut}} = 0.2$, so each Mutator has a 20% chance to fire each tick. When it fires, the agent:

(1) picks a card s_1 uniformly at random from the cards it actually owns,
(2) picks a different card s_2 uniformly from the other two card types,
(3) updates its deck so that $d_{s_1} \leftarrow d_{s_1} - 1$ and $d_{s_2} \leftarrow d_{s_2} + 1$, and
(4) posts a sell order for one s_1 and a buy order for one s_2, both at `limit_price` = 1.0.

Picking s_1 first and then a different s_2 matches the branching $A \xrightarrow{\mu} \{B, C\}$ structure in Eq. (2): it is symmetric across all pairs and never picks the same card twice, so on average the Mutator does not favor any particular card. It just adds randomness and exploration to the system, the same role μ plays in the ODE.

5.2.2 Copycat The Copycat is our agent-level version of the reaction in Eq. (1), but framed as imitation: a player who lost is more likely to copy a player who beat them. Each tick, the agent looks at its most recent game record (result, opp_uid, $\mathbf{d}_{\text{opp}}$) and the current population $\mathbf{c}$, and uses Eq. (8) to estimate both its own fitness and its opponent's:

$$f_{\text{self}} = \mathbf{x}(\mathbf{d})^\top P\, \mathbf{x}(\mathbf{c}), \tag{10}$$

$$f_{\text{opp}} = \mathbf{x}(\mathbf{d}_{\text{opp}})^\top P\, \mathbf{x}(\mathbf{c}). \tag{11}$$

The probability that the agent copies the opponent's deck is

$$p_{\text{copy}} \;=\; \min\!\left(1,\; \max(0,\, f_{\text{opp}} - f_{\text{self}}) \;\cdot\; \frac{z}{\mu N}\right), \tag{12}$$

where z is the copycat rate. Again, the agent uniformly samples a value in $(0.0, 0.99)$, and if the sample is less than p_{copy}, the agent overwrites its deck with $\mathbf{d}_{\text{opp}}$; otherwise it does nothing.

A few things to note about Eq. (12):

(1) *Only copies stronger decks.* The $\max(0, \cdot)$ piece means the player never imitates an opponent whose deck is worse than their own, so imitation always pushes toward higher fitness.
(2) *Scales with population size.* The $z/(\mu N)$ factor ties the per-agent imitation rate to the ratio z/μ from the ODE, so Copycat and Mutator events stay in the same proportion regardless of how many players we run.
(3) *Capped at 1.* The outer $\min(1, \cdot)$ keeps the result a valid probability even when the fitness gap is large.

Unlike the Mutator, the Copycat does not place any market orders: it just copies the opponent's deck directly. We made this choice on purpose, because in real life a player who sees a winning deck will usually try to clone the whole thing rather than slowly trade toward it one card at a time. Keeping imitation off the market

also makes our results easier to interpret, since any difference between the simulation and the ODE baseline can be blamed on Mutator trades and scalper activity rather than on slippage in the imitation channel.

5.3 Proving every edge in the graph lies on a directed cycle

Problem Setup We model the card market as a directed graph G on n vertices (players). Each player is assigned a label (b, s) where $b \in \{\mathrm{R}, \mathrm{P}, \mathrm{S}\}$ is the card they wish to buy and $s \in \{\mathrm{R}, \mathrm{P}, \mathrm{S}\}$ is the card they have to sell. A directed edge from Player i to Player j indicates that Player i is buying from Player j, which requires $s_j = b_i$: the card Player j sells must match the card Player i is buying.

Since each player submits exactly one buy order and exactly one sell order:

- Each vertex has out-degree exactly 1 (one outgoing edge, representing the unique counterparty from whom they buy).
- Each vertex has in-degree exactly 1 (one incoming edge, representing the unique counterparty who buys from them).

These are the structural constraints. The specific Rock–Paper–Scissors labels impose an additional consistency requirement on adjacent labels, but the cycle structure follows purely from the degree constraint.

Main Theorem

Theorem 5.1. *Let G be a finite directed graph on n vertices in which every vertex has in-degree equal to out-degree equal to* 1*. Then every edge of G lies on a directed cycle.*

Proof. Let $e = (u_0, u_1)$ be an arbitrary edge of G. We will show that e is contained in a directed cycle.

Constructing a forward path. Starting from u_1, since every vertex has out-degree 1, there is a unique edge leaving u_1; let its head be u_2. Continuing inductively, for each $k \geq 1$ let u_{k+1} be the unique successor of u_k. This produces an infinite sequence of vertices

$$u_0 \to u_1 \to u_2 \to u_3 \to \cdots$$

Applying the Pigeonhole Principle. Since G has only n vertices, the infinite sequence $u_0, u_1, u_2, \ldots$ cannot consist of pairwise distinct vertices. Hence there exist indices $j < k$ such that $u_k = u_j$. Let k be the smallest such index, and let j be the corresponding (necessarily unique) earlier index with $u_k = u_j$. We just need to prove $j = 0$ and we are done.

Showing $j = 0$. Suppose for contradiction that $j \geq 1$. Then $u_k = u_j$, so vertex u_j has at least two incoming edges:

$$(u_{j-1}, u_j) \quad \text{and} \quad (u_{k-1}, u_k) = (u_{k-1}, u_j).$$

By minimality of k, all vertices $u_0, u_1, \ldots, u_{k-1}$ are pairwise distinct, so in particular $u_{j-1} \neq u_{k-1}$. These are therefore two *distinct* edges entering u_j, giving u_j in-degree at least 2. This contradicts the assumption that every vertex has in-degree exactly 1.

Therefore $j = 0$, which means $u_k = u_0$.

Conclusion. The sequence

$$u_0 \to u_1 \to u_2 \to \cdots \to u_{k-1} \to u_0$$

is a directed cycle of length k in G. By construction, this cycle contains the edge $e = (u_0, u_1)$. Since e was arbitrary, every edge of G lies on a directed cycle. □

Remark In the card-game context, this means the market always self-organizes into disjoint trading rings, regardless of which buy/sell orders the players submit, as long as the constraints are satisfied.

6 Experimental Results and Validation

6.1 Experimental Procedure

To validate our simulator, we used the baseline ODE solutions from the **Simulation** section. Our experiments consist of two study types: a standalone ODE solve and a side-by-side simulation-vs-ODE comparison. Both are implemented as pytest tests in `tests/test_plots.py` and share the same parameter set from `config.py`. Table 3 shows the parameter values used for our validation.

ODE baseline (`test_ode_plot`). We run `OdeRPS.SolveODE` with the initial conditions derived from `SEED_DECK_WEIGHTS` (Rock:Paper:Scissors = 5:2:1, normalized to fractions) over $t \in [0, 10]$ with 500 uniformly-spaced evaluation points. The result is shown in Figure 1 and shows how the three card populations evolve over time under the Szolnoki ODEs alone, with no scalper agent or market noise.

Table 3: Simulation parameters used in all experiments.

Parameter	Value
Players (N)	500
Ticks	10,000
Copycat rate (z)	1.1
Mutation rate (μ)	0.01
RNG seed	442
ODE time span	$[0, \text{NUM_TICKS}/N] = [0, 10]$
ODE scale (s)	16
ODE eval points	500

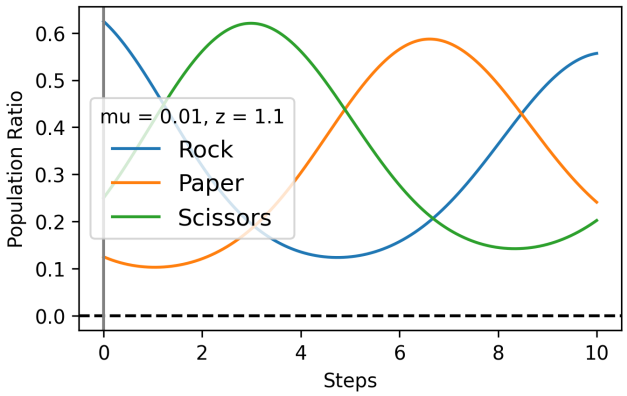


Figure 1: Szolnoki ODE Plot

Simulation vs. ODE (`test_comparison_plot`). To validate the simulation against the ODE, we first instantiate a simulation with $N = 500$ Copycat players, 0 scalpers, and `GameNineCard` as the game type. Before running it, we snapshot the initial card counts from the simulation and pass them directly to `OdeRPS` as initial conditions, so both systems start from the exact same fractions. We then run the simulation for 5,000 ticks and the ODE solver for $t \in [0, T_{\text{end}} \cdot s]$ where

$$T_{\text{end}} = \frac{\text{NUM_TICKS}}{\text{NUM_PLAYERS}} = \frac{5000}{500} = 10 \tag{13}$$

is the equivalent ODE time span, and $s = 16$ (`ODE_SCALE`) is a stretch factor that lets the ODE trajectory fully develop its oscillations. The plot can be seen in Figure 2. The resulting comparison plot has two panels: the top shows the ODE solution, and the bottom shows the raw simulation data overlaid with its cumulative time average.

6.2 Validating the Simulator Against the ODE

The purpose of the comparison test is to check that our agent-based simulation, in the absence of scalpers, reproduces the same qualitative population dynamics as the ODE baseline. The comparison test uses a mixed population of Copycat and Mutator players allocated in the same ratio as the ODE rate constants: a fraction $z/(z+\mu)$ Copycats and $\mu/(z+\mu)$ Mutators. With $z = 1.1$ and $\mu = 0.01$ this works out to roughly 99.1% Copycats and 0.9% Mutators, which directly mirrors the balance of the copycat and mutation terms in the ODE. The results of the comparison test is shown in

Time axis alignment. The simulation operates in discrete ticks, while the ODE operates in continuous time. To compare them on the same axis, simulation tick k is mapped to ODE time $t = k/N$, where N is the number of players. This rescaling reflects the mean-field argument: one "ODE time unit" corresponds to N agent interactions on average, analogous to one generation in a population dynamics model.

Cumulative time average. Because individual simulation ticks are noisy (each tick, only a fraction of players act), we compute the cumulative time average of each card's population fraction:

$$\bar{x}_k = \frac{1}{k} \sum_{i=1}^{k} x_i, \tag{14}$$

where x_i is the population fraction of a given card at tick i. The raw per-tick values are plotted at 25% opacity in the background, and the cumulative average is plotted as a dashed line on top. As k grows, $\bar{x}_k$ converges toward the time-mean of the ODE trajectory, providing a stable signal to compare against.

Quantitative error metrics. For a finer-grained comparison, the `ErrorVisualizer` interpolates the ODE solution onto the simulation's time grid using `numpy.interp`, then computes per-card signed error, RMSE, and MAE between the cumulative average and the ODE:

$$\text{RMSE} = \sqrt{\frac{1}{K} \sum_{k=1}^{K} \left(\bar{x}_k - x_k^{\text{ODE}} \right)^2}, \tag{15}$$

$$\text{MAE} = \frac{1}{K} \sum_{k=1}^{K} \left| \bar{x}_k - x_k^{\text{ODE}} \right|. \tag{16}$$

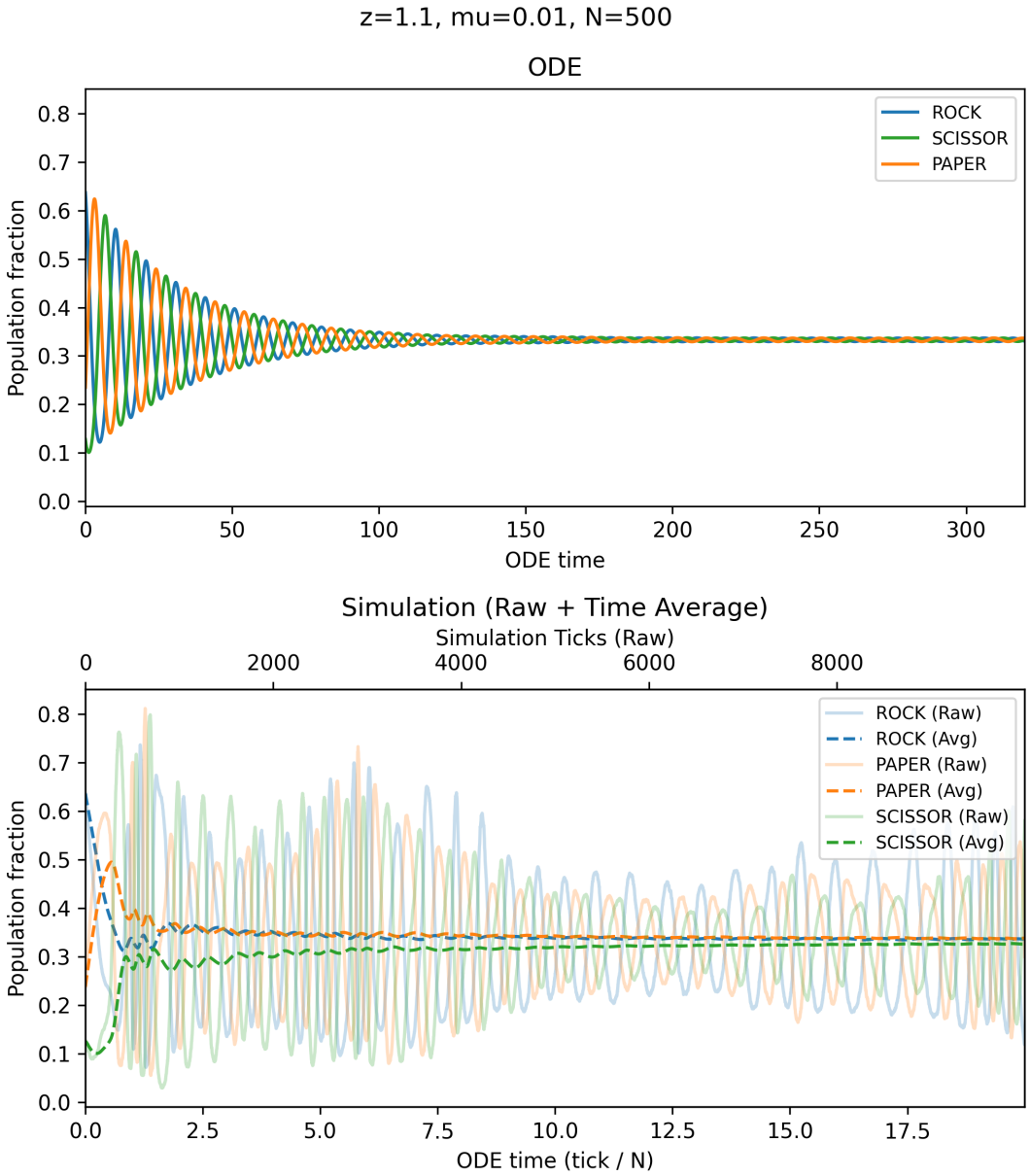


Figure 2: Plots of Card Population Fractions – ODE vs Simulation

All time axes are normalized to $[0, 1]$ before computing error so that results are not sensitive to the choice of T_{end} or s.

Initial condition justification: Rather than picking a synthetic starting point, we seed the ODE directly from the simulation's own initial card distribution. This means any observed difference between the two trajectories comes from simulation dynamics, not from a mismatch in starting conditions.

6.2.1 Analysis of Results When reviewing Figure 2, we can see our ODE baseline comparison at work. The upper subplot is the expected trajectory of the simulation, and the lower subplot shows one stochastic realization of the system. By observation, we can see that both the ODE baseline and the simulation's cumulative time

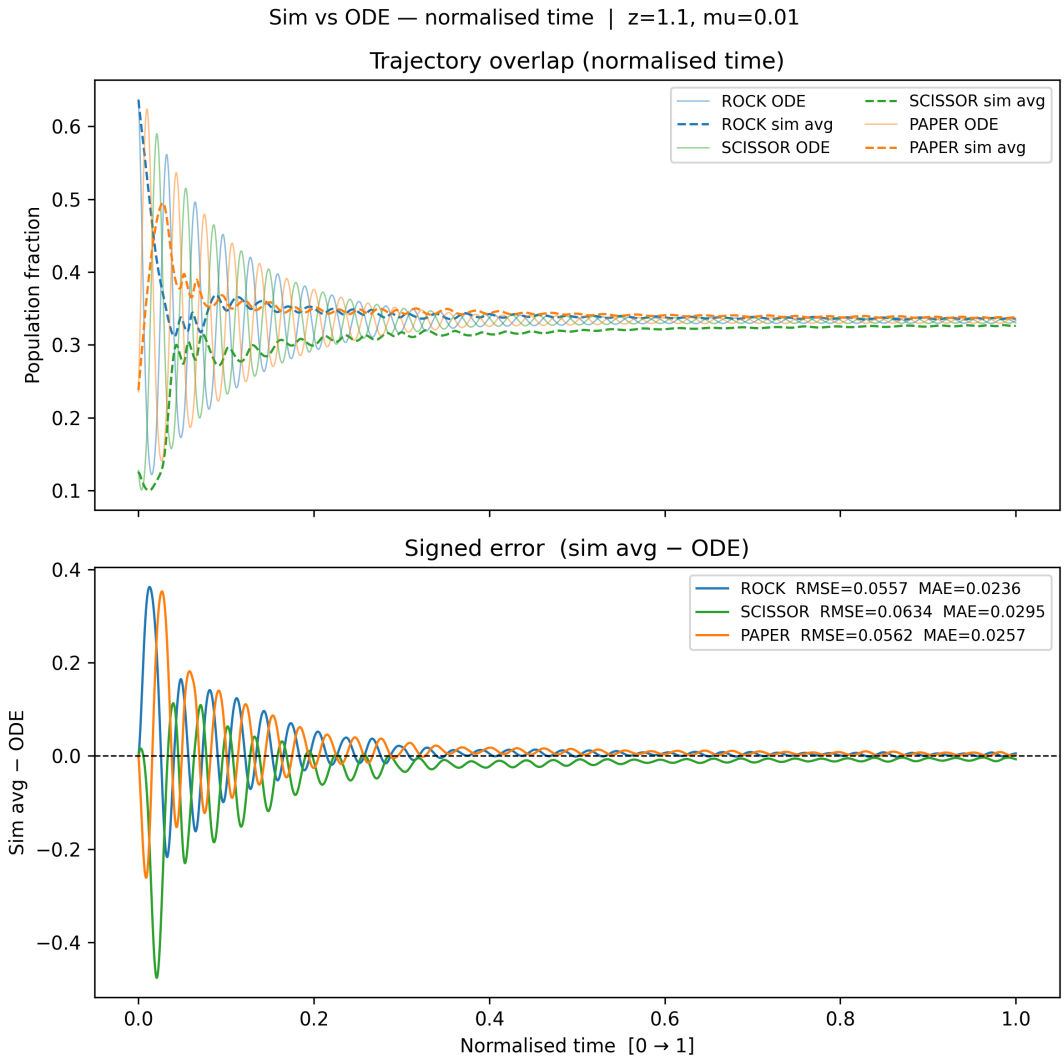


Figure 3: Plots of Trajectory overlap and Error Estimation – ODE vs Simulation

average card population values tend towards the same trajectory at large step counts. Both are nearing the expected steady-state of $R = P = S \approx 0.333\ldots$.

6.2.2 On Simulation Noise One of the key findings of this project is the noisy nature of the simulation's results. When looking at the raw population curves in Figure 2, we can see that even at large step counts, raw card populations exhibit large variance and never converge as tightly as the ODE result. This is likely due to the underlying nature of the ODE approach. The ODE baseline is a mean-field representation that assumes a perfectly-mixed, continuous system. By contrast, our simulation uses 500 agents and discrete time intervals. This is why we validate the simulation using the cumulative time average of the card populations.

Threshold justification and results. The thresholds `MAX_RMSE` = 0.08 and `MAX_MAE` = 0.08 were chosen to be generous enough to tolerate the stochastic noise inherent in a single run of $N = 500$ agents over 5,000 ticks, while still being tight enough to catch a systematic implementation bug. Because the population fractions always sum to 1 and there are three cards, a maximally wrong model could achieve RMSE ≈ 0.33; our threshold of 0.08 therefore rejects anything that deviates by more than roughly one-quarter of the total range. Table 4 shows

the measured values from the run that corresponds to Figure 3.

Card	RMSE	MAE	Max |err|
Rock	0.0557	0.0236	0.3626
Scissor	0.0634	0.0295	0.4762
Paper	0.0562	0.0257	0.3531

Table 4: Per-card normalized-time error: sim cumulative average vs. ODE trajectory. All values pass the 0.08 threshold.

The RMSE and MAE values are well below the threshold, confirming that the cumulative average tracks the ODE shape closely. The large Max |err| values (0.36–0.48) are not contradictory: they occur early in the run when only a handful of ticks have been observed and the simulation is in a highly noisy state. By the time the simulation is past its early ticks, the signed error in Figure 3 drops and remains close to zero, hovering symmetrically above and below the zero line rather than drifting persistently in one direction. That zero-mean, symmetric pattern is the key indicator that the simulation is behaving as intended: it tells us the stochastic noise is unbiased, not a sign of a systematic bug that consistently over- or under-produces one card type relative to what the ODE predicts.

7 Discussion

7.1 Agent Based Simulation

Our project demonstrates a foundation for a coupled trading card game and market simulation framework. We have demonstrated that agent based simulation can be numerically validated according to published research. This project highlighted the stochastic nature of agent-based simulations. It challenged us to troubleshoot and effectively implement numerical methods, which proved to be a very iterative process. In multiple rounds of on-the-fly testing, we found that the simulations can be fragile given certain population fractions. If μ and z fell out of certain ranges, the simulation would collapse to unexpected card populations.

7.2 Scalper Performance

To evaluate scalper strategies under varied but reproducible conditions, we ran 20 independent simulations, deploying two agents of each type per run, yielding 40 samples per strategy. Each scalper began with $10. Table 5 reports average final balances.

Table 5: Average final balance per scalper type across 20 runs (40 samples each).

Agent	Avg. Final Balance ($)
NaiveGambler	9.1510
Shadow	10.1138
QuickGrabber	9.9228
MarketMaker	10.2684

Table 6 presents all six pairwise comparisons, showing the mean difference in final balance and the winning strategy in each matchup.

Table 6: Pairwise comparisons of scalper strategies.

Matchup	Δ($)	Winner
Naive vs. Shadow	+0.96	Shadow
Naive vs. QuickGrabber	+0.77	QuickGrabber
Naive vs. MarketMaker	+1.12	MarketMaker
Shadow vs. QuickGrabber	+0.19	Shadow
Shadow vs. MarketMaker	+0.15	MarketMaker
Quick vs. MarketMaker	+0.35	MarketMaker

The overall ranking is MarketMaker > Shadow > QuickGrabber > NaiveGambler. MarketMaker's contrarian entry rule—buying the card with the steepest recent price drop—most consistently exploits mean reversion in this market, outperforming both the EMA-based Shadow strategy and the momentum-driven QuickGrabber. Crucially, all three structured strategies outperform the random baseline, confirming that even simple signal-driven rules extract value from the simulated price dynamics.

7.3 Future Work

7.3.1 Analytics and Reporting We would like to add post-simulation reporting with CSV export so that researchers can run their own downstream analysis on the raw data.

7.3.2 Interface and Visualization The simulation currently runs in the terminal. A real-time interactive dashboard would make it easier to visualize price trajectories, scalper positions, and population dynamics as they unfold.

7.3.3 Player Trading Strategies In the current implementation, players do not participate in the card market. A Copycat player could observe the deck composition of a recent opponent and purchase cards to mirror a winning strategy, while a Mutator player could periodically swap a card in its deck for a randomly chosen alternative, introducing controlled noise into the population mix. Enabling player-side trading would create a richer feedback loop. Game outcomes would influence market demand, and market activity would in turn reshape the decks that enter future games.

7.3.4 Scalability and Performance The current simulation stores agents in Python lists and iterates over them sequentially each tick. For experiments involving hundreds or thousands of agents, vectorizing the agent update step with NumPy, or batching order processing within the market, would substantially reduce time.

7.3.5 Reproducibility and Experiment Management While the simulation supports seeded random number generators for reproducibility, there is currently no structured way to record experiment configurations, replay specific runs, or compare results across parameter settings. Integrating a experiment-tracking framework would make it easier for researchers to reproduce findings and build on each other's work.

8 AI Statement

(1) OpenAI ChatGPT was used for finding and aggregating relevant research papers. It was also used for formatting bib references. All papers listed above were read and distilled into this literature review by a human.
(2) Anthropic Claude was used for generating UML diagrams based on human descriptions and code references.
(3) Anthropic Claude was used to generate unit tests by scanning the codebase. All produced code was thoroughly reviewed by a human.
(4) Anthropic Claude was used to run the scalper code 20 times and aggregate the results.
(5) Anthropic Claude was used for generating plotting/visualization code and error estimation code.
(6) Anthropic Claude was used to style and comment code to enable easy comprehension by other teammates.
(7) GPT 5.3 was used for Latex formatting and language editing.

References

[1] William A. Brock and Cars H. Hommes. 1998. Heterogeneous Beliefs and Routes to Chaos in a Simple Asset Pricing Model. *Journal of Economic Dynamics and Control* 22, 8–9 (1998), 1235–1274. doi:10.1016/S0165-1889(98)00011-6

[2] Pascal Courty. 2003. Some Economics of Ticket Resale. *Journal of Economic Perspectives* 17, 2 (2003), 85–97. doi:10.1257/089533003765888458

[3] J. R. Dormand and P. J. Prince. 1980. A family of embedded Runge-Kutta formulae. *J. Comput. Appl. Math.* 6, 1 (1980), 19–26. doi:10.1016/0771-050X(80)90013-3

[4] Dhananjay K. Gode and Shyam Sunder. 1993. Allocative Efficiency of Markets with Zero-Intelligence Traders: Market as a Partial Substitute for Individual Rationality. *Journal of Political Economy* 101, 1 (1993), 119–137. doi:10.1086/261868

[5] E. Hairer, S. P. Nørsett, and G. Wanner. 1993. *Solving Ordinary Differential Equations I: Nonstiff Problems* (2 ed.). Springer-Verlag, Berlin.

[6] Hamidreza Moradi, Erfan Kefayat, and Hamideh Hossei. 2025. Investigation of the Estimation Accuracy of 5 Different Numerical ODE Solvers on 3 Case Studies. *arXiv preprint arXiv:2502.10289* (2025).

[7] William L. Silber. 1984. Marketmaker Behavior in an Auction Market: An Analysis of Scalpers in Futures Markets. *The Journal of Finance* 39, 4 (1984), 937–953. doi:10.2307/2327606

[8] Attila Szolnoki, Mauro Mobilia, Luo-Luo Jiang, Bartosz Szczesny, Alastair M. Rucklidge, and Matjaž Perc. 2014. Cyclic Dominance in Evolutionary Games: A Review. *Journal of the Royal Society Interface* 11, 100 (2014), 20140735. doi:10.1098/rsif.2014.0735

[9] Leigh Tesfatsion. 2002. Agent-based computational economics: growing economies from the bottom up. *Artificial Life* 8, 1 (2002), 55–82. doi:10.1162/106454602753694765

Appendix A

Division of Labor

Table 7 provides an overview of how the work was divided. Each group member contributed equally to the project.

Table 7: Division of labor among team members.

Component	Owner(s)
Simulation loop & architecture	Adrian, Denys
Game logic	Denys
Player agents	Denys
Scalper agents	Smaran
Market & order matching	Adrian, Smaran
Baseline ODE & numerical methods	Denys, Smaran
Visualization & reporting	Adrian, Denys
Report writing	All